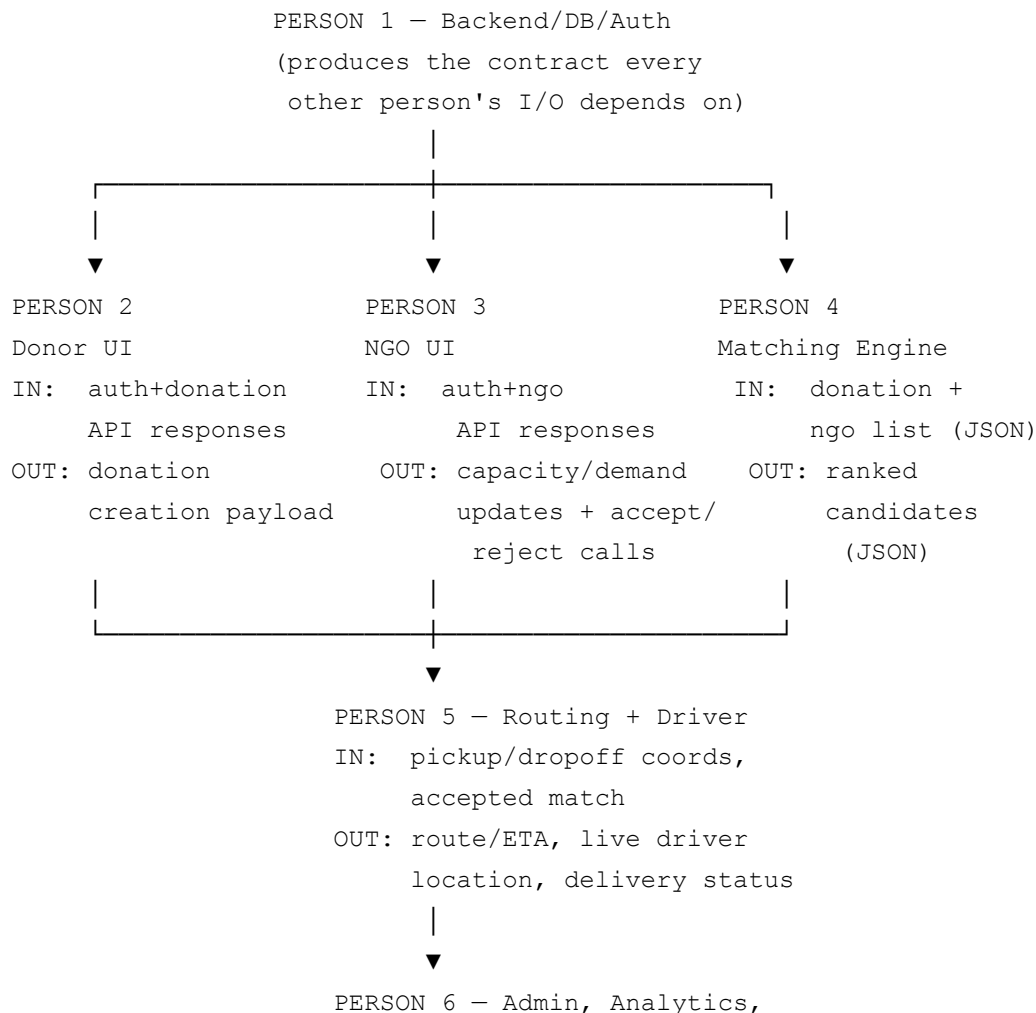


Food-Waste Routing Platform

API Contract Organized by Person — Inputs, Outputs & Integration Ownership

This is the version of the contract each of the six people should actually work from. Instead of reading the whole API top to bottom, each person gets: what they **receive** (and from whom), what they must **produce** (and who consumes it), the exact JSON shape on both sides, and a **mock fixture** they can build against on Day 1 without waiting on anyone else. The last section assigns who owns final integration.

0. How the six people connect (read this first)



```
Integration & Final Assembly
IN:  everything above
OUT: KPIs, verified NGOs,
      the merged, deployed system
```

Nobody talks to anybody's database directly. Every arrow above is an HTTP call or a WebSocket event defined in this document — that's the entire point of freezing a contract.

1. Global conventions (all six people use these — no exceptions)

- Base URL: `/api/v1/...`
 - Auth header: `Authorization: Bearer <jwt>` on everything except `/auth/register`, `/auth/login`, `/auth/refresh`.
 - Success envelope: `{ "data": {...}, "meta": { "request_id": "..." } }`
 - List envelope adds: `"meta": { "next_cursor": "...", "has_more": true }`
 - Error envelope: `{ "error": { "code": "...", "message": "...", "field": null, "request_id": "..." } }`
 - Idempotency: any mutating call that a flaky connection might retry (`accept`, `pickup`, `deliver`) requires header `Idempotency-Key: <uuid>`.
 - All timestamps UTC ISO-8601 (`...Z`). All weights in kg, float, 1 decimal.
-

2. PERSON 1 — Backend, Database & Authentication

Role in the pipeline: foundation. Produces the contract everyone else's input/output depends on. Consumes nothing from the other five — this is why Person 1 must ship first, even if only against mock data internally.

What Person 1 must output, and who consumes it

Endpoint	Method	Consumed by	Output shape
<code>/auth/register</code>	POST	2, 3, 5 (via their login screens)	<code>{ "data": { "user_id", "role", "verification_status" } }</code>

<code>/auth/login</code>	POST	2, 3, 5	<pre>{ "data": { "access_token", "refresh_token", "expires_in", "user": {...} } }</pre>
<code>/auth/refresh</code>	POST	all frontend clients	new <code>access_token</code> + rotated <code>refresh_token</code>
<code>/auth/me</code>	GET	2, 3, 5, 6	current user profile + linked <code>donor_id / ngo_id / driver_id</code>
<code>/donations</code> (CRUD)	POST/GET/PATCH	2 (writes), 4 (reads), 6 (reads)	see §3 for shape — Person 1 owns the table, Person 2 owns the form that fills it
<code>/ngos</code> (CRUD)	GET/PATCH	3 (writes capacity/demand), 4 (reads for filtering), 6 (reads for verification)	see §4
<code>/drivers ,</code> <code>/deliveries ,</code> <code>/handover</code> tables	—	5 writes, 1 stores, 6 reads	Person 1 creates the schema; Person 5 is the only one who calls the write endpoints day-to-day
WebSocket infra <code>/ws/donations ,</code> <code>/ws/deliveries ,</code> <code>/ws/drivers</code>	—	2, 3, 5 subscribe	Person 1 stands up the broker; everyone else just subscribes and emits through it

What Person 1 needs as input

Nothing from the other five to *start*. Needs from Person 4 by Day 3: the exact shape of a "match result" object (already fixed below in §5, so this isn't a blocker) so the `donations` table has the right columns (`matched_ngo_id`, `match_score`, `weights_version_id`).

Day-1 deliverable (before anyone else can build against real endpoints)

- Running FastAPI skeleton + PostgreSQL/PostGIS + Docker Compose
- `/auth/*` fully working

- Empty-but-schema-correct CRUD stubs for `/donations`, `/ngos`, `/drivers`, `/deliveries` returning realistic **mock JSON** matching the shapes in this document, even before real logic exists — this is what unblocks 2, 3, 4, 5 to build in parallel starting Day 1, not Day 5.

Definition of Done

PostgreSQL + PostGIS running · JWT auth working · all table schemas from §§3–8 migrated · every endpoint below returns contract-correct JSON (mocked or real) · Docker Compose brings up the whole stack in one command · Postman/OpenAPI collection published so 2–6 can hit real endpoints without reading code.

3. PERSON 2 — Donor Application

Role in the pipeline: produces the donation that everything downstream depends on.

Input Person 2 receives (and from whom)

From	What	Shape
Person 1	Auth tokens, donation CRUD responses	<code>GET /donations/{id}</code> → full object below
Person 4 (via Person 1's API)	Match status for donor's own donation	<code>GET /matching/{donation_id}/candidates</code> (read-only for donor: donor sees <i>which</i> NGO was matched, not full score breakdown)
Person 5 (via Person 1's API / WebSocket)	Live driver location, ETA	<code>/ws/deliveries</code> → <code>delivery.location_update</code>

Output Person 2 must produce (and who consumes it)

This is the single most important object in the whole system — Person 4 cannot score anything without it, Person 1 cannot store anything without it:

```
// POST /donations — Person 2's output, consumed by Person 1 (storage) → Person
{
  "food_name": "Vegetable Rice",
```

```

    "food_category": "COOKED",
    "quantity_kg": 35.0,
    "prepared_at": "2026-09-09T08:00:00Z",
    "available_from": "2026-09-09T12:00:00Z",
    "expiry_time": "2026-09-09T15:00:00Z",
    "pickup_location": { "latitude": 12.9352, "longitude": 77.6245, "address": "12,
    "special_handling": "Keep refrigerated below 5°C",
    "food_safety_info": {
      "storage_temp_required": "REFRIGERATED",
      "allergen_tags": ["dairy"],
      "packaging_type": "SEALED_CONTAINER"
    }
  }
}

```

Response Person 2 must correctly render (this is what feeds the donor dashboard):

```

{
  "data": {
    "id": "don_9931",
    "status": "AVAILABLE",
    "matchedngo_id": null,
    "driver_id": null,
    "eta_minutes": null,
    "created_at": "2026-09-09T08:05:00Z"
  }
}

```

Other outputs Person 2 owns:

Endpoint	Consumed by
PATCH /donations/{id}/cancel { "reason": "..."} }	Person 1 (state change) → Person 4 (removes from candidate pool if still matching)
POST /donations/{id}/photos (multipart)	Person 6 (shows in audit trail)

Mock fixture to build against on Day 1 (no dependency on Person 1)

donations_mock.json — 5 sample donations in every status (AVAILABLE , MATCHED , IN_TRANSIT , DELIVERED , EXPIRED) so the dashboard, lifecycle badges, and map view can all be built before the real backend responds.

Definition of Done

Login → create donation form posts the exact shape above · dashboard renders all 9 lifecycle statuses correctly · donor map shows donor→driver→NGO with live ETA · donor can cancel pre-pickup · donor sees final handover confirmation.

4. PERSON 3 — NGO / Recipient Module

Role in the pipeline: the demand side — without accurate capacity/demand, Person 4's scoring is meaningless.

Input Person 3 receives

From	What	Shape
Person 1	Auth, NGO profile CRUD	<code>GET /ngos/{id}</code>
Person 4 (via Person 1's API)	Incoming donation offers, ranked	<code>GET /ngos/{id}/incoming</code> → array of donation summaries with <code>eta_minutes</code> , <code>remaining_shelf_life_min</code> , <code>distance_km</code>
Person 5	Driver ETA once accepted	<code>/ws/deliveries</code> → <code>delivery.driver_assigned</code>

Output Person 3 must produce

```
// PATCH /ngos/{id}/capacity — consumed by Person 4 (hard-constraint filter input
{ "available_capacity_kg": 72.0 }
```

```
// PATCH /ngos/{id}/demand — consumed by Person 4 (demand score D)
{
  "food_category": "COOKED",
  "required_quantity_kg": 30,
  "priority": "HIGH",
  "valid_until": "2026-09-09T20:00:00Z"
}
```

```
// POST /matching/{donation_id}/accept — consumed by Person 1 (state transition)
// Header required: Idempotency-Key
{ "ngo_id": "ngo_017" }
```

```
// POST /matching/{donation_id}/reject — consumed by Person 4 (triggers rematch)
{ "ngo_id": "ngo_017", "reason": "Insufficient capacity today" }

// POST /handover/{delivery_id} — NGO's half of digital sign-off — consumed by Pe
{
  "ngo_confirmation": true,
  "quantity_handed_over": 33.5,
  "condition": "GOOD",
  "notes": "2 crates slightly delayed but within safe window"
}
```

Mock fixture to build against on Day 1

`ngos_mock.json` + `incoming_donations_mock.json` — 3 NGOs with varying capacity/demand, and a mock incoming-offer payload, so the accept/reject screen and capacity dashboard work before Person 1's or Person 4's real logic exists.

Definition of Done

NGO registration + capacity/demand management working · incoming-donation screen renders `eta_minutes`, `distance_km`, `remaining_shelf_life_min` correctly · accept/reject calls fire with idempotency key · handover confirmation screen posts the exact shape above.

5. PERSON 4 — Matching & Optimisation Engine

Role in the pipeline: the differentiator. Pure backend logic — no UI. This person's whole job is: take Person 2's donation + Person 1's live NGO table, produce a ranked list Person 3 and Person 1 can act on.

Input Person 4 receives

```
// Input A — a donation object (from Person 1's DB, originated by Person 2)
{
  "id": "don_9931",
  "food_category": "COOKED",
  "quantity_kg": 35.0,
  "pickup_location": { "latitude": 12.9352, "longitude": 77.6245 },
  "available_from": "2026-09-09T12:00:00Z",
  "expiry_time": "2026-09-09T15:00:00Z"
}
```

```

}

// Input B – candidate NGO list (from Person 1's DB, maintained by Person 3)
[
  {
    "ngo_id": "ngo_017",
    "location": { "latitude": 12.9345, "longitude": 77.6104 },
    "available_capacity_kg": 72.0,
    "accepted_categories": ["COOKED", "PACKAGED"],
    "operating_hours": { "start": "08:00", "end": "20:00" },
    "is_verified": true,
    "demand": { "food_category": "COOKED", "required_quantity_kg": 30, "priority"
  }
]

// Input C – a route/ETA per candidate (from Person 5, called internally by Perso
{ "ngo_id": "ngo_017", "distance_km": 5.4, "duration_minutes": 21 }

```

Output Person 4 must produce (this exact shape is what Persons 1 and 3 build their screens against)

```

// GET /matching/{donation_id}/candidates – consumed by Person 3 (NGO offer scree
{
  "data": {
    "donation_id": "don_9931",
    "weights_version_id": "wv_2026_bengaluru_cooked_v3",
    "matches": [
      {
        "ngo_id": "ngo_017",
        "score": 0.91,
        "capacity_score": 0.94,
        "shelf_life_score": 0.87,
        "transit_score": 0.90,
        "demand_score": 1.0,
        "route_score": 0.83,
        "eta_minutes": 17
      },
      { "ngo_id": "ngo_009", "score": 0.76, "eta_minutes": 11 }
    ]
  }
}

```

If no candidate survives hard-constraint filtering:

```
{ "data": { "donation_id": "don_9931", "weights_version_id": "wv_2026_bengaluru_c
```

— consumed by Person 1, which sets donation status to `NO_MATCH_FOUND` and notifies Person 2's donor UI.

Mock fixtures — Person 4 builds entirely without waiting on anyone

`donations.json` + `ngos.json` sample files (Day-1 deliverable per the base plan) — Person 4 should never be blocked by Person 1's or Person 3's UI progress. Deliverables: `candidate_filter.py`, `scoring.py`, `optimizer.py`, `vrp.py` (stretch), `test_matching.py`.

Definition of Done

Hard-constraint filter matches the 5-step chain exactly · all five scores (C, S, T, D, R) implemented and unit-tested · weighted ranking configurable per `weights_version_id` · automatic rematch on reject · output JSON matches the shape above byte-for-byte (this is the part that breaks integration if it drifts).

6. PERSON 5 — Routing, Dispatch & Driver Application

Role in the pipeline: turns an accepted match into an actual movement of food.

Input Person 5 receives

From	What	Shape
Person 4 (via Person 1)	Accepted match — pickup + dropoff pair	{ "pickup": {...}, "dropoff": {...}, "quantity_kg": 35 }
Person 1	Driver pool	GET /drivers?status=AVAILABLE

Output Person 5 must produce

```
// POST /routes/calculate — consumed internally by Person 4 (route_score input) a
{
  "data": { "distance_km": 5.4, "duration_minutes": 21, "geometry": "encoded_poly
}
```

```
// POST /drivers/location — rate-limited 1 req/5 sec — consumed by Person 1 (broa
{ "latitude": 12.934, "longitude": 77.620 }
```

```
// POST /deliveries/{id}/pickup — Header: Idempotency-Key — consumed by Person 1
{ "confirmed_quantity_kg": 35.0 }
```

```
// POST /deliveries/{id}/deliver — Header: Idempotency-Key — consumed by Person 1
{ "quantity_handed_over": 33.5, "condition": "GOOD", "recipient_confirmation": tr
```

Mock fixture to build against on Day 1

`sample_coordinates.json` — origin/destination pairs across Bengaluru so the routing proof-of-concept and driver UI skeleton don't wait on a live matching engine.

Definition of Done

Routing API returns distance/ETA/geometry · driver dashboard shows current job · live location broadcasts every ≤ 5 sec · pickup/deliver endpoints fire with idempotency keys · dispatch logic (NGO accepted $\rightarrow$ find driver $\rightarrow$ assign $\rightarrow$ route) runs end-to-end.

7. PERSON 6 — Admin, Analytics, Integration & Final Assembly

Role in the pipeline: consumes everything the other five produce, and — per this task — owns pulling the whole system together into one deployed, demo-ready product.

Input Person 6 receives (from everyone)

From	What
Person 1	Full DB access for admin views, audit logs
Person 2	Donation volume/status for food KPIs
Person 3	NGO verification requests, capacity/demand history
Person 4	<code>weights_version_id</code> history for explainability, match outcomes for logistics KPIs
Person 5	Delivery timestamps, route distances for logistics KPIs and route-distance-saved calculation

Output Person 6 must produce

```
// GET /analytics/overview — consumed by admin dashboard (Person 6's own UI)
{
  "data": {
    "total_donations": 214,
    "total_food_rescued_kg": 3120.5,
    "active_donations": 6,
    "active_deliveries": 2,
    "registered_ngos": 41,
    "registered_donors": 28,
    "available_drivers": 5
  }
}
```

```
// GET /analytics/food | /logistics | /social — consumed by the impact dashboard
{
  "data": {
    "kg_diverted": 3120.5,
    "meals_recovered": 6241,
    "delivery_success_rate": 0.94,
    "avg_matching_time_sec": 4.2,
    "avg_delivery_time_min": 26,
    "route_distance_saved_km": 118.3,
    "organisations_served": 41,
    "beneficiaries_reached": 9800
  }
}
```

```
// PATCH /admin/ngos/{id}/verify — consumed by Person 3 (gates the NGO out of mat
{ "status": "APPROVED", "reason": "Registration certificate + FSSAI Alliance ID v
```

Integration & final assembly — assigned explicitly

Person 6 is the Integration Lead, with **Person 1 as Integration Co-Lead** (Person 1 owns the schema everything else is built on, so merge conflicts almost always trace back to a backend contract question only Person 1 can resolve fast). This isn't "sit around doing admin CRUD" — it's the job of making sure the other four modules actually snap together into one working system.

Concretely, Person 6 owns:

1. **The monorepo and branch hygiene** — enforcing `feature/* → develop → main`, blocking direct pushes to `main`, running the API-contract diff check on every PR that touches a shared endpoint.
2. **The mock-to-real cutover schedule** — deciding, module by module, when Person 2/3/5 stop pointing at mock JSON and start hitting Person 1's real endpoints (this should happen incrementally, not all six switching over the night before the demo).
3. **The integration test suite** (`tests/integration/`) — the six scenarios already defined in the base plan (capacity rejection, expiry rejection, category rejection, rematch-on-reject, concurrent-accept race, expiry-during-matching) plus one end-to-end scenario that exercises all six people's code in one run:

```
Person 2 creates donation
→ Person 1 stores it
→ Person 4 scores candidates
→ Person 3's NGO accepts
→ Person 5 assigns driver, routes, delivers
→ Person 3 confirms handover
→ Person 1 writes the audit record
→ Person 6's analytics reflect the new totals
```

This single test passing end-to-end is the actual "definition of done" for the whole project, not any one person's individual checklist.

4. **Docker Compose assembly** — one `docker-compose up` that brings up frontend, backend, Postgres, Redis, and the matching worker together; Person 6 is responsible for this working on a laptop that isn't any of the six people's own dev machine.
5. **Deployment** — whatever target is chosen (a single cloud VM, Render/Railway, or a local demo box), Person 6 does the final deploy and smoke-tests every one of the API endpoints in this document against the deployed instance, not just localhost.
6. **Demo script ownership** — Person 6 runs the rehearsal of the exact demo scenario from the base plan (Donor 1, NGO A/B/C, live scoring, route, delivery, sign-off, analytics update) at least twice before presentation day, on the deployed build, not a dev machine.

Why Person 6 and not, say, Person 4 (the "most important technical contributor"):

Person 4's time is worth protecting for the algorithm, which is the project's actual differentiator — pulling them into merge-conflict resolution and deployment babysitting is the single most common way student teams end up with a brilliant algorithm that never actually runs end-to-end on demo day.

Definition of Done

All KPI endpoints live and correct · NGO verification workflow gates matching eligibility · integration test suite (6 unit scenarios + 1 end-to-end scenario) passing in CI · one-command Docker Compose bring-up verified on a clean machine · deployed instance smoke-tested against every endpoint in this document · demo rehearsed twice on the deployed build.

8. Cross-person freeze rule

Once this document is agreed (target: end of Day 2), any change to a request/response shape in §§2–7 requires:

1. A PR against this document itself, not a silent code change.
2. Sign-off from the person who *owns* the changed endpoint **and** the person(s) listed as consumers of it.
3. A changelog entry below.

Example of what this prevents: Person 4 silently renaming `score` to `match_score` in the candidates response, which breaks Person 3's NGO offer screen without anyone finding out until integration week.

Changelog

- `v1` — initial per-person contract freeze, integration ownership assigned to Person 6 (lead) / Person 1 (co-lead).